

Chapter 10. Cross-Cluster Data Mirroring

For most of the book we discuss the setup, maintenance, and use of a single Kafka cluster. There are, however, a few scenarios in which an architecture may need more than one cluster.

In some cases, the clusters are completely separated. They belong to different departments or different use cases, and there is no reason to copy data from one cluster to another. Sometimes, different SLAs or workloads make it difficult to tune a single cluster to serve multiple use cases. Other times, there are different security requirements. Those use cases are fairly easy—managing multiple distinct clusters is the same as running a single cluster multiple times.

In other use cases, the different clusters are interdependent, and the administrators need to continuously copy data between the clusters. In most databases, continuously copying data between database servers is called *replication*. Since we've used replication to describe movement of data between Kafka nodes that are part of the same cluster, we'll call copying of data between Kafka clusters *mirroring*. Apache Kafka's built-in cross-cluster replicator is called *MirrorMaker*.

In this chapter, we will discuss cross-cluster mirroring of all or part of the data. We'll start by discussing some of the common use cases for cross-cluster mirroring. Then we'll show a few architectures that are used to implement these use cases and discuss the pros and cons of each architecture pattern. We'll then discuss MirrorMaker itself and how to use it. We'll share operational tips, including deployment and performance tuning. We'll finish by discussing a few alternatives to MirrorMaker.

Use Cases of Cross-Cluster Mirroring

The following is a list of examples of when cross-cluster mirroring would be used:

Regional and central clusters

In some cases, the company has one or more datacenters in different geographical regions, cities, or continents. Each datacenter has its own Kafka cluster. Some applications can work just by communicating with the local cluster, but some applications require data from multiple datacenters (otherwise, you wouldn't be looking at cross-datacenter replication solutions). There are many cases when this is a requirement, but the classic example is a company that modifies prices based on supply and demand. This company can have a datacenter in each city in which it has a presence, collects information about local supply and demand, and adjusts prices accordingly. All this information will then be mirrored to a central cluster where business analysts can run company-wide reports on its revenue.

High availability (HA) and disaster recovery (DR)

The applications run on just one Kafka cluster and don't need data from other locations, but you are concerned about the possibility of the entire cluster becoming unavailable for some reason. For redundancy, you'd like to have a second Kafka cluster with all the data that exists in the first cluster, so in case of emergency you can direct your applications to the second cluster and continue as usual.

Regulatory compliance

Companies operating in different countries may need to use different configurations and policies to conform to legal and regulatory requirements in each country. For instance, some datasets may be stored in separate clusters with strict access control, with subsets of data replicated to other clusters with wider access. To comply with regulatory policies that govern retention period in each region, datasets may be stored in clusters in different regions with different configurations.

Cloud migrations

Many companies these days run their business in both an on-premises datacenter and a cloud provider. Often, applications run on multiple regions of the cloud provider for redundancy, and sometimes multiple cloud providers are used. In these cases, there is often at least one Kafka cluster in each on-premises datacenter and each cloud region. Those Kafka clusters are used by applications in each datacenter and region to transfer data efficiently between the datacenters. For example, if a new application is deployed in the cloud but requires some data that is updated by applications running in the on-premises datacenter and stored in an on-premises database, you can use Kafka Connect to capture database changes to the local Kafka cluster and then mirror these changes to the cloud Kafka cluster where the new application can use them. This helps control the costs of cross-datacenter traffic as well as improve governance and security of the traffic.

Aggregation of data from edge clusters

Several industries, including retail, telecommunications, transportation, and healthcare, generate data from small devices with limited connectivity. An aggregate cluster with high availability can be used to support analytics and other use cases for data from a large number of edge clusters. This reduces connectivity, availability, and durability requirements on low-footprint edge clusters, for example, in IoT use cases. A highly available aggregate cluster provides business continuity even when edge clusters are offline and simplifies the development of applications that don't have to directly deal with a large number of edge clusters with unstable networks.

Multicluster Architectures

Now that we've seen a few use cases that require multiple Kafka clusters, let's look at some common architectural patterns that we've successfully used when implementing these use cases. Before we go into the architectures, we'll give a brief overview of the realities of cross-datacenter communications. The solutions we'll discuss may seem overly complicated without understanding that they represent trade-offs in the face of specific network conditions.

Some Realities of Cross-Datacenter Communication

The following is a list of some things to consider when it comes to cross-datacenter communication:

High latencies

Latency of communication between two Kafka clusters increases as the distance and the number of network hops between the two clusters increase.

Limited bandwidth

Wide area networks (WANs) typically have far lower available bandwidth than what you'll see inside a single datacenter, and the available bandwidth can vary from minute to minute. In addition, higher latencies make it more challenging to utilize all the available bandwidth.

Higher costs

Regardless of whether you are running Kafka on premise or in the cloud, there are higher costs to communicate between clusters. This is partly because the bandwidth is limited and adding bandwidth can be prohibitively expensive, and also because of the prices vendors charge for transferring data among datacenters, regions, and clouds.

Apache Kafka's brokers and clients were designed, developed, tested, and tuned, all within a single datacenter. We assumed low latency and high bandwidth between brokers and clients. This is apparent in the default timeouts and sizing of various buffers. For this reason, it is not recommended (except in specific cases, which we'll discuss later) to install some Kafka brokers in one datacenter and others in another datacenter.

In most cases, it's best to avoid producing data to a remote datacenter, and when you do, you need to account for higher latency and the potential for more network errors. You can handle the errors by increasing the number of producer retries, and handle the higher latency by increasing the size of the buffers that hold records between attempts to send them.

If we need any kind of replication between clusters, and we ruled out inter-broker communication and producer-broker communication, then we must allow for broker-consumer communication. Indeed, this is the safest form of cross-cluster communication because in the event of network partition that prevents a consumer from reading data, the records remain safe inside the Kafka brokers until communications resume and consumers can read them. There is no risk of accidental data loss due to network partitions. Still, because bandwidth is limited, if there are multiple applications in one datacenter that need to read data from Kafka brokers in another datacenter, we prefer to install a Kafka cluster in each datacenter and mirror the necessary data between them once rather than have multiple applications consume the same data across the WAN.

We'll talk more about tuning Kafka for cross-datacenter communication, but the following principles will guide most of the architectures we'll discuss next:

- No less than one cluster per datacenter.
- Replicate each event exactly once (barring retries due to errors) between each pair of datacenters.

- When possible, consume from a remote datacenter rather than produce to a remote datacenter.

Hub-and-Spoke Architecture

This architecture is intended for the case where there are multiple local Kafka clusters and one central Kafka cluster. See [Figure 10-1](#).

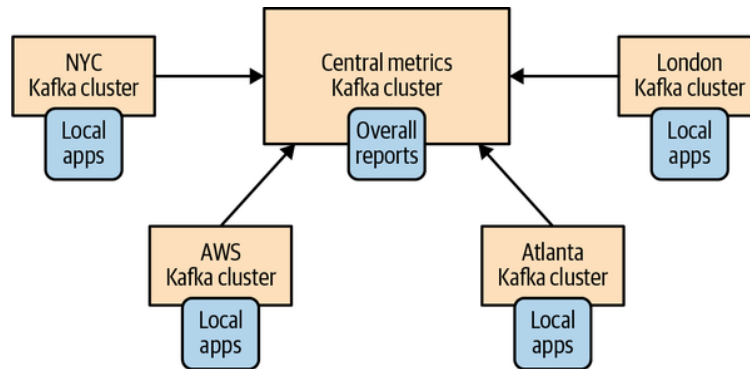


Figure 10-1. The hub-and-spoke architecture

There is also a simpler variation of this architecture with just two clusters: a leader and a follower. See [Figure 10-2](#).

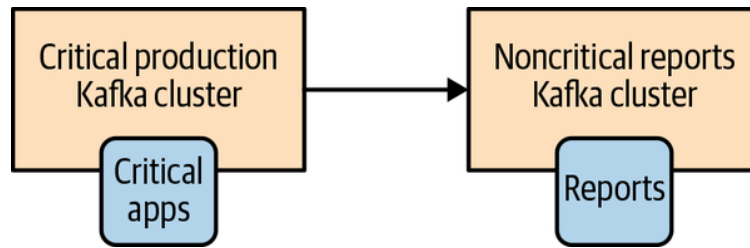


Figure 10-2. A simpler version of the hub-and-spoke architecture

This architecture is used when data is produced in multiple datacenters and some consumers need access to the entire dataset. The architecture also allows for applications in each datacenter to only process data local to that specific datacenter. But it does not give access to the entire dataset from every datacenter.

The main benefit of this architecture is that data is always produced to the local datacenter and events from each datacenter are only mirrored once—to the central datacenter. Applications that process data from a single datacenter can be located at that datacenter. Applications that need to process data from multiple datacenters will be located at the central datacenter where all the events are mirrored. Because replication always goes in one direction and because each consumer always reads from the same cluster, this architecture is simple to deploy, configure, and monitor.

The main drawback of this architecture is the direct result of its benefits and simplicity. Processors in one regional datacenter can't access data in another. To understand better why this is a limitation, let's look at an example of this architecture.

Suppose that we are a large bank and have branches in multiple cities. Let's say that we decide to store user profiles and their account history in a Kafka cluster in each city. We replicate all this information to a central cluster that is used to run the bank's business analytics. When users connect to the bank website or visit their local branch, they are routed to send events to their local cluster and read events from the same local cluster. However, suppose that a user visits a branch in a different city. Because the user information doesn't exist in the city they are visiting, the

branch will be forced to interact with a remote cluster (not recommended) or have no way to access the user's information (really embarrassing). For this reason, use of this pattern is usually limited to only parts of the dataset that can be completely separated between regional datacenters.

When implementing this architecture, for each regional datacenter you need at least one mirroring process on the central datacenter. This process will consume data from each remote regional cluster and produce it to the central cluster. If the same topic exists in multiple datacenters, you can write all the events from this topic to one topic with the same name in the central cluster, or write events from each datacenter to a separate topic.

Active-Active Architecture

This architecture is used when two or more datacenters share some or all of the data, and each datacenter is able to both produce and consume events. See [Figure 10-3](#).

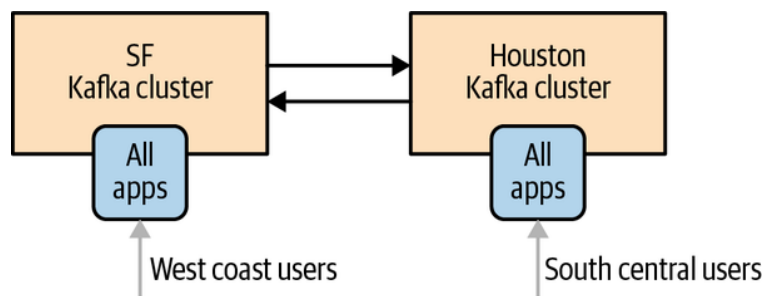


Figure 10-3. The active-active architecture model

The main benefit of this architecture is the ability to serve users from a nearby datacenter, which typically has performance benefits, without sacrificing functionality due to limited availability of data (as we've seen happen in the hub-and-spoke architecture). A secondary benefit is redundancy and resilience. Since every datacenter has all the functionality, if one datacenter is unavailable, you can direct users to a remaining datacenter. This type of failover only requires network redirects of users, typically the easiest and most transparent type of failover.

The main drawback of this architecture is the challenge in avoiding conflicts when data is read and updated asynchronously in multiple locations. This includes technical challenges in mirroring events—for example, how do we make sure the same event isn't mirrored back and forth endlessly? But more importantly, maintaining data consistency between the two datacenters will be difficult. Here are few examples of the difficulties you will encounter:

- If a user sends an event to one datacenter and reads events from another datacenter, it is possible that the event they wrote hasn't arrived at the second datacenter yet. To the user, it will look like they just added a book to their wish list and clicked on the wish list, but the book isn't there. For this reason, when this architecture is used, developers usually find a way to "stick" each user to a specific datacenter and make sure they use the same cluster most of the time (unless they connect from a remote location or the datacenter becomes unavailable).
- An event from one datacenter says the user ordered book A, and an event from more or less the same time at a second datacenter says that the same user ordered book B. After mirroring, both datacenters have both events and thus we can say that each datacenter has two conflict-

ing events. Applications on both datacenters need to know how to deal with this situation. Do we pick one event as the “correct” one? If so, we need consistent rules on how to pick one event so applications on both datacenters will arrive at the same conclusion. Do we decide that both are true and simply send the user two books and have another department deal with returns? Amazon used to resolve conflicts that way, but organizations dealing with stock trades, for example, can’t. The specific method for minimizing conflicts and handling them when they occur is specific to each use case. It is important to keep in mind that if you use this architecture, you *will* have conflicts and will need to deal with them.

If you find ways to handle the challenges of asynchronous reads and writes to the same dataset from multiple locations, then this architecture is highly recommended. It is the most scalable, resilient, flexible, and cost-effective option we are aware of. So, it is well worth the effort to figure out solutions for avoiding replication cycles, keeping users mostly in the same datacenter, and handling conflicts when they occur.

Part of the challenge of active-active mirroring, especially with more than two datacenters, is that you will need mirroring tasks for each pair of datacenters and each direction. Many mirroring tools these days can share processes, for example, using the same process for all mirroring to a destination cluster.

In addition, you will want to avoid loops in which the same event is mirrored back and forth endlessly. You can do this by giving each “logical topic” a separate topic for each datacenter and making sure to avoid replicating topics that originated in remote datacenters. For example, logical topic *users* will be topic *SF.users* in one datacenter and *NYC.users* in another datacenter. The mirroring processes will mirror topic *SF.users* from SF to NYC and topic *NYC.users* from NYC to SF. As a result, each event will only be mirrored once, but each datacenter will contain both *SF.users* and *NYC.users*, which means each datacenter will have information for all the users. Consumers will need to consume events from **.users* if they wish to consume all user events. Another way to think of this setup is to see it as a separate namespace for each datacenter that contains all the topics for the specific datacenter. In our example, we’ll have the NYC and the SF namespaces. Some mirroring tools like MirrorMaker prevent replication cycles using a similar naming convention.

Record headers introduced in Apache Kafka in version 0.11.0 enable events to be tagged with their originating datacenter. Header information may also be used to avoid endless mirroring loops and to allow processing events from different datacenters separately. You can also implement this feature by using a structured data format for the record values (Avro is our favorite example) and use this to include tags and headers in the event itself. However, this does require extra effort when mirroring, since none of the existing mirroring tools will support your specific header format.

Active-Standby Architecture

In some cases, the only requirement for multiple clusters is to support some kind of disaster scenario. Perhaps you have two clusters in the same datacenter. You use one cluster for all the applications, but you want a second cluster that contains (almost) all the events in the original cluster that you can use if the original cluster is completely unavailable. Or perhaps you need geographic resiliency. Your entire business is running from a datacenter in California, but you need a second datacenter in

Texas that usually doesn't do much and that you can use in case of an earthquake. The Texas datacenter will probably have an inactive ("cold") copy of all the applications that admins can start up in case of emergency and that will use the second cluster (Figure 10-4). This is often a legal requirement rather than something that the business is actually planning on doing—but you still need to be ready.

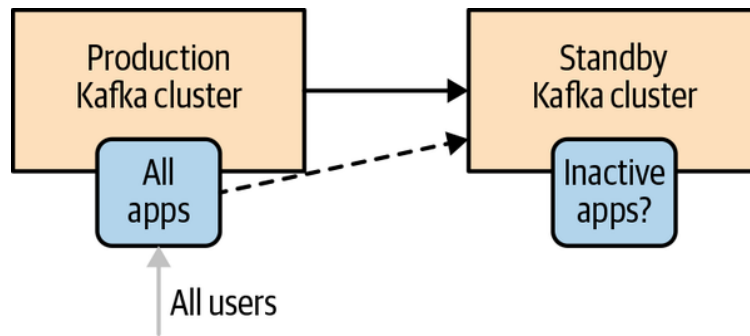


Figure 10-4. The active-standby architecture

The benefits of this setup are simplicity in setup and the fact that it can be used in pretty much any use case. You simply install a second cluster and set up a mirroring process that streams all the events from one cluster to another. No need to worry about access to data, handling conflicts, and other architectural complexities.

The disadvantages are waste of a good cluster and the fact that failover between Kafka clusters is, in fact, much harder than it looks. The bottom line is that it is currently not possible to perform cluster failover in Kafka without either losing data or having duplicate events. Often both. You can minimize them but never fully eliminate them.

It should be obvious that a cluster that does nothing except wait around for a disaster is a waste of resources. Since disasters are (or should be) rare, most of the time we are looking at a cluster of machines that does nothing at all. Some organizations try to fight this issue by having a DR (disaster recovery) cluster that is much smaller than the production cluster. But this is a risky decision because you can't be sure that this minimally sized cluster will hold up during an emergency. Other organizations prefer to make the cluster useful during nondisasters by shifting some read-only workloads to run on the DR cluster, which means they are really running a small version of a hub-and-spoke architecture with a single spoke.

The more serious issue is, how do you failover to a DR cluster in Apache Kafka?

First, it should go without saying that whichever failover method you choose, your SRE team must practice it on a regular basis. A plan that works today may stop working after an upgrade, or perhaps new use cases make the existing tooling obsolete. Once a quarter is usually the bare minimum for failover practices. Strong SRE teams practice far more frequently. Netflix's famous Chaos Monkey, a service that randomly causes disasters, is the extreme—any day may become failover practice day.

Now, let's take a look at what is involved in a failover.

Disaster recovery planning

When planning for disaster recovery, it is important to consider two key metrics. Recovery time objective (RTO) defines the maximum amount of time before all services must resume after a disaster. Recovery point ob-

jective (RPO) defines the maximum amount of time for which data may be lost as a result of a disaster. The lower the RTO, the more important it is to avoid manual processes and application restarts, since very low RTO can be achieved only with automated failover. Low RPO requires real-time mirroring with low latencies, and $RPO=0$ requires synchronous replication.

Data loss and inconsistencies in unplanned failover

Because Kafka's various mirroring solutions are all asynchronous (we'll discuss a synchronous solution in the next section), the DR cluster will not have the latest messages from the primary cluster. You should always monitor how far behind the DR cluster is and never let it fall too far behind. But in a busy system you should expect the DR cluster to be a few hundred or even a few thousand messages behind the primary. If your Kafka cluster handles 1 million messages a second and the lag between the primary and the DR cluster is 5 milliseconds, your DR cluster will be 5,000 messages behind the primary in the best-case scenario. So, prepare for unplanned failover to include some data loss. In planned failover, you can stop the primary cluster and wait for the mirroring process to mirror the remaining messages before failing over applications to the DR cluster, thus avoiding this data loss. When unplanned failover occurs and you lose a few thousand messages, note that mirroring solutions currently don't support transactions, which means that if some events in multiple topics are related to each other (e.g., sales and line items), you can have some events arrive to the DR site in time for the failover and others that don't. Your applications will need to be able to handle a line item without a corresponding sale after you failover to the DR cluster.

Start offset for applications after failover

One of the challenging tasks in failing over to another cluster is making sure applications know where to start consuming data. There are several common approaches. Some are simple but can cause additional data loss or duplicate processing; others are more involved but minimize additional data loss and reprocessing. Let's take a look at a few:

Auto offset reset

Apache Kafka consumers have a configuration for how to behave when they don't have a previously committed offset—they either start reading from the beginning of the partition or from the end of the partition. If you are not somehow mirroring these offsets as part of the DR plan, you need to choose one of these options. Either start reading from the beginning of available data and handle large amounts of duplicates or skip to the end and miss an unknown (and hopefully small) number of events. If your application handles duplicates with no issues, or missing some data is no big deal, this option is by far the easiest. Simply skipping to the end of the topic on failover is a popular failover method due to its simplicity.

Replicate offsets topic

If you are using Kafka consumers from version 0.9.0 and later, the consumers will commit their offsets to a special topic:

`__consumer_offsets`. If you mirror this topic to your DR cluster, when consumers start consuming from the DR cluster, they will be able to pick up their old offsets and continue from where they left off. It is simple, but there is a long list of caveats involved.

First, there is no guarantee that offsets in the primary cluster will match those in the secondary cluster. Suppose you only store data in the primary cluster for three days and you start mirroring a topic a week after it was created. In this case, the first offset available in the primary cluster may be offset 57,000,000 (older events were from the first 4 days and were removed already), but the first offset in the DR cluster will be 0. So, a consumer that tries to read offset 57,000,003 (because that's its next event to read) from the DR cluster will fail to do this.

Second, even if you started mirroring immediately when the topic was first created and both the primary and the DR topics start with 0, producer retries can cause offsets to diverge. We discuss an alternative mirroring solution that preserves offsets between primary and DR clusters at the end of this chapter.

Third, even if the offsets were perfectly preserved, because of the lag between primary and DR clusters and because mirroring solutions currently don't support transactions, an offset committed by a Kafka consumer may arrive ahead or behind the record with this offset. A consumer that fails over may find committed offsets without matching records. Or it may find that the latest committed offset in the DR site is older than the latest committed offset in the primary site. See [Figure 10-5](#).

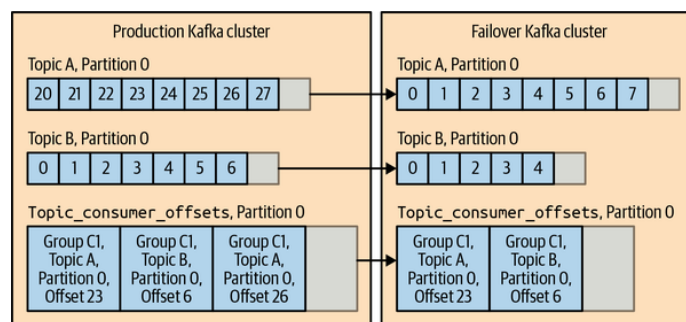


Figure 10-5. A failover causes committed offsets without matching records

In these cases, you need to accept some duplicates if the latest committed offset in the DR site is older than the one committed on the primary or if the offsets in the records in the DR site are ahead of the primary due to retries. You will also need to figure out how to handle cases where the latest committed offset in the DR site doesn't have a matching record—do you start processing from the beginning of the topic or skip to the end?

As you can see, this approach has its limitations. Still, this option lets you failover to another DR with a reduced number of duplicated or missing events compared to other approaches while still being simple to implement.

Time-based failover

From version 0.10.0 onward, each message includes a timestamp indicating the time the message was sent to Kafka. From 0.10.1.0 onward, brokers include an index and an API for looking up offsets by the timestamp. So, if you failover to the DR cluster and you know that your trouble started at 4:05 a.m., you can tell consumers to start processing data from 4:03 a.m. There will be some duplicates from those two minutes, but it is probably better than other alternatives and the behavior is much easier to explain to everyone in the company—"We failed back to 4:03 a.m." sounds better than "We failed back to what may or may not be the latest committed

offsets.” So, this is often a good compromise. The only question is: how do we tell consumers to start processing data from 4:03 a.m.?

One option is to bake it right into your app. Have a user-configurable option to specify the start time for the app. If this is configured, the app can use the new APIs to fetch offset by time, seek to that time, and start consuming from the right point, committing offsets as usual.

This option is great if you wrote all your applications this way in advance. But what if you didn’t? Apache Kafka provides the `kafka-consumer-groups` tool to reset offsets based on a range of options, including timestamp-based reset added in 0.11.0. The consumer group should be stopped while running this type of tool and started immediately after. For example, the following command resets consumer offsets for all topics belonging to a particular group to a specific time:

```
bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --reset-offsets --al
```

This option is recommended in deployments that need to guarantee a level of certainty in their failover.

Offset translation

When discussing mirroring the offsets topic, one of the biggest challenges is the fact that offsets in primary and DR clusters can diverge. In the past, some organizations chose to use an external data store, such as Apache Cassandra, to store mapping of offsets from one cluster to another. Whenever an event is produced to the DR cluster, both offsets are sent to the external data store by the mirroring tool when offsets diverge. These days, mirroring solutions, including MirrorMaker, use a Kafka topic for storing offset translation metadata. Offsets are stored whenever the difference between the two offsets changes. For example, if offset 495 on the primary mapped to offset 500 on the DR cluster, we’ll record (495,500) in the external store or offset translation topic. If the difference changes later due to duplicates and offset 596 is mapped to 600, then we’ll record the new mapping (596,600). There is no need to store all the offset mappings between 495 and 596; we just assume that the difference remains the same and so offset 550 in the primary cluster will map to 555 in the DR. Then when failover occurs, instead of mapping timestamps (which are always a bit inaccurate) to offsets, we map primary offsets to DR offsets and use those. One of the two techniques listed previously can be used to force consumers to start using the new offsets from the mapping. This still has an issue with offset commits that arrived ahead of the records themselves and offset commits that didn’t get mirrored to the DR on time, but it covers some cases.

After the failover

Let’s say that failover was successful. Everything is working just fine on the DR cluster. Now we need to do something with the primary cluster. Perhaps turn it into a DR.

It is tempting to simply modify the mirroring processes to reverse their direction and simply start mirroring from the new primary to the old one. However, this leads to two important questions:

- How do we know where to start mirroring? We need to solve the same problem we have for all our consumers for the mirroring application itself. And remember that all our solutions have cases where they either cause duplicates or miss data—sometimes both.
- In addition, for reasons we discussed previously, it is likely that your original primary will have events that the DR cluster does not. If you just start mirroring new data back, the extra history will remain and the two clusters will be inconsistent.

For this reason, for scenarios where consistency and ordering guarantees are critical, the simplest solution is to first scrape the original cluster—delete all the data and committed offsets—and then start mirroring from the new primary back to what is now the new DR cluster. This gives you a clean slate that is identical to the new primary.

A few words on cluster discovery

One of the important points to consider when planning a standby cluster is that in the event of failover, your applications will need to know how to start communicating with the failover cluster. If you hardcoded the host-names of your primary cluster brokers in the producer and consumer properties, this will be challenging. Most organizations keep it simple and create a DNS name that usually points to the primary brokers. In case of an emergency, the DNS name can be pointed to the standby cluster. The discovery service (DNS or other) doesn't need to include all the brokers—Kafka clients only need to access a single broker successfully in order to get metadata about the cluster and discover the other brokers. So, including just three brokers is usually fine. Regardless of the discovery method, most failover scenarios do require bouncing consumer applications after failover so they can find the new offsets from which they need to start consuming. For automated failover without application restart to achieve very low RTO, failover logic should be built into client applications.

Stretch Clusters

Active-standby architectures are used to protect the business against the failure of a Kafka cluster by moving applications to communicate with another cluster in case of cluster failure. Stretch clusters are intended to protect the Kafka cluster from failure during a datacenter outage. This is achieved by installing a single Kafka cluster across multiple datacenters.

Stretch clusters are fundamentally different from other multidatacenter scenarios. To start with, they are not multicluster—it is just one cluster. As a result, we don't need a mirroring process to keep two clusters in sync. Kafka's normal replication mechanism is used, as usual, to keep all brokers in the cluster in sync. This setup can include synchronous replication. Producers normally receive an acknowledgment from a Kafka broker after the message was successfully written to Kafka. In the stretch cluster case, we can configure things so the acknowledgment will be sent after the message is written successfully to Kafka brokers in two datacenters. This involves using rack definitions to make sure each partition has replicas in multiple datacenters, and the use of `min.insync.replicas` and `acks=all` to ensure that every write is acknowledged from at least two datacenters. From 2.4.0 onward, brokers can also be configured to enable consumers to fetch from the closest replica using rack definitions. Brokers match their rack with that of the consumer to find the local replica that is most up-to-date, falling back to the leader if a suitable local replica is not available. Consumers fetching from followers in their local

datacenter achieve higher throughput, lower latency, and lower cost by reducing cross-datacenter traffic.

The advantages of this architecture are in the synchronous replication—some types of business simply require that their DR site is always 100% synchronized with the primary site. This is often a legal requirement and is applied to any data store across the company—Kafka included. The other advantage is that both datacenters and all brokers in the cluster are used. There is no waste like we saw in active-standby architectures.

This architecture is limited in the type of disasters it protects against. It only protects from datacenter failures, not any kind of application or Kafka failures. The operational complexity is also limited. This architecture demands physical infrastructure that not all companies can provide.

This architecture is feasible if you can install Kafka (and ZooKeeper) in at least three datacenters with high bandwidth and low latency between them. This can be done if your company owns three buildings on the same street, or—more commonly—by using three availability zones inside one region of your cloud provider.

The reason three datacenters are important is because ZooKeeper requires an uneven number of nodes in a cluster and will remain available if a majority of the nodes are available. With two datacenters and an uneven number of nodes, one datacenter will always contain a majority, which means that if this datacenter is unavailable, ZooKeeper is unavailable, and Kafka is unavailable. With three datacenters, you can easily allocate nodes so no single datacenter has a majority. So, if one datacenter is unavailable, a majority of nodes exist in the other two datacenters, and the ZooKeeper cluster will remain available. Therefore, so will the Kafka cluster.

2.5 DC ARCHITECTURE

A popular model for stretch clusters is a 2.5 DC (datacenter) architecture with both Kafka and ZooKeeper running in two datacenters, and a third “0.5” datacenter with one ZooKeeper node to provide quorum if a datacenter fails.

It is possible to run ZooKeeper and Kafka in two datacenters using a ZooKeeper group configuration that allows for manual failover between two datacenters. However, this setup is uncommon.

Apache Kafka’s MirrorMaker

Apache Kafka contains a tool called MirrorMaker for mirroring data between two datacenters. Early versions of MirrorMaker used a collection of consumers that were members of a consumer group to read data from a set of source topics and a shared Kafka producer in each MirrorMaker process to send those events to the destination cluster. While this was sufficient to mirror data across clusters in some scenarios, it had several issues, particularly latency spikes as configuration changes and addition of new topics resulted in stop-the-world rebalances. MirrorMaker 2.0 is the next-generation multicluster mirroring solution for Apache Kafka that is based on the Kafka Connect framework, overcoming many of the shortcomings of its predecessor. Complex topologies can be easily configured to support a wide range of use cases like disaster recovery, backup, migration, and data aggregation.

MirrorMaker sounds very simple, but because we were trying to be very efficient and get very close to exactly-once delivery, it turned out to be tricky to implement correctly. MirrorMaker has been rewritten multiple times. The description here and the details in the following sections apply to MirrorMaker 2.0, which was introduced in 2.4.0.

MirrorMaker uses a source connector to consume data from another Kafka cluster rather than from a database. Use of the Kafka Connect framework minimizes administration overhead for busy enterprise IT departments. If you recall the Kafka Connect architecture from [Chapter 9](#), you remember that each connector divides the work among a configurable number of tasks. In MirrorMaker, each task is a consumer and a producer pair. The Connect framework assigns those tasks to different Connect worker nodes as needed—so you may have multiple tasks on one server or have the tasks spread out to multiple servers. This replaces the manual work of figuring out how many MirrorMaker streams should run per instance and how many instances per machine. Connect also has a REST API to centrally manage the configuration for the connectors and tasks. If we assume that most Kafka deployments include Kafka Connect for other reasons (sending database change events into Kafka is a very popular use case), then by running MirrorMaker inside Connect, we can cut down on the number of clusters we need to manage.

MirrorMaker allocates partitions to tasks evenly without using Kafka's consumer group-management protocol to avoid latency spikes due to rebalances when new topics or partitions are added. Events from each partition in the source cluster are mirrored to the same partition in the target cluster, preserving semantic partitioning and maintaining ordering of events for each partition. If new partitions are added to source topics, they are automatically created in the target topic. In addition to data replication, MirrorMaker also supports migration of consumer offsets, topic configuration, and topic ACLs, making it a complete mirroring solution for multicluster deployments. A replication flow defines the configuration of a directional flow from a source cluster to a target cluster. Multiple replication flows can be defined for MirrorMaker to define complex topologies, including the architectural patterns we discussed earlier like hub-and-spoke, active-standby, and active-active architectures.

[Figure 10-6](#) shows the use of MirrorMaker in an active-standby architecture.

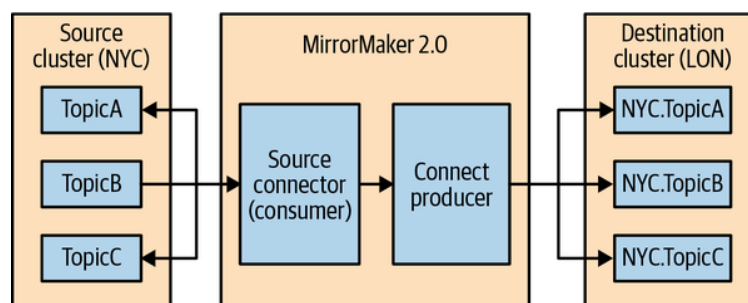


Figure 10-6. The MirrorMaker process in Kafka

Configuring MirrorMaker

MirrorMaker is highly configurable. In addition to the cluster settings to define the topology, Kafka Connect, and connector settings, every configuration property of the underlying producer, consumers, and admin client used by MirrorMaker can be customized. We will show a few examples

here and highlight some of the important configuration options, but exhaustive documentation of MirrorMaker is outside our scope.

With that in mind, let's take a look at a MirrorMaker example. The following command starts MirrorMaker with the configuration options specified in the properties file:

```
bin/connect-mirror-maker.sh etc/kafka/connect-mirror-maker.properties
```

Let's look at some of MirrorMaker's configuration options:

Replication flow

The following example shows the configuration options for setting up an active-standby replication flow between two datacenters in New York and London:

```
clusters = NYC, LON                                ❶  
NYC.bootstrap.servers = kafka.nyc.example.com:9092 ❷  
LON.bootstrap.servers = kafka.lon.example.com:9092  
NYC->LON.enabled = true                           ❸  
NYC->LON.topics = .*                               ❹
```

- ❶ Define aliases for the clusters used in replication flows.
- ❷ Configure bootstrap for each cluster, using the cluster alias as the prefix.
- ❸ Enable replication flow between a pair of clusters using the prefix `source->target`. All configuration options for this flow use the same prefix.
- ❹ Configure the topics to be mirrored for this replication flow.

Mirror topics

As shown in the example, for each replication flow, a regular expression may be specified for the topic names that will be mirrored. In this example, we chose to replicate every topic, but it is often good practice to use something like *prod.** and avoid replicating test topics. A separate topic exclusion list containing topic names or patterns like *test.** may also be specified to exclude topics that don't require mirroring. Target topic names are automatically prefixed with the source cluster alias by default. For example, in active-active architecture, MirrorMaker replicating topics from an NYC datacenter to a LON datacenter will mirror the topic *orders* from NYC to the topic *NYC.orders* in LON. This default naming strategy prevents replication cycles resulting in events being endlessly mirrored between the two clusters in active-active mode if topics are mirrored from NYC to LON as well as LON to NYC. The distinction between local and remote topics also supports aggregation use cases since consumers may choose subscription patterns to consume data produced from just the local region or subscribe to topics from all regions to get the complete dataset.

MirrorMaker periodically checks for new topics in the source cluster and starts mirroring these topics automatically if they match the configured patterns. If more partitions are added to the source topic, the same number of partitions is automatically added to the

target topic, ensuring that events in the source topic appear in the same partitions in the same order in the target topic.

Consumer offset migration

MirrorMaker contains a utility class `RemoteClusterUtils` to enable consumers to seek to the last checkpointed offset in a DR cluster with offset translation when failing over from a primary cluster. Support for periodic migration of consumer offsets was added in 2.7.0 to automatically commit translated offsets to the target `__consumer_offsets` topic so that consumers switching to a DR cluster can restart from where they left off in the primary cluster with no data loss and minimal duplicate processing. Consumer groups for which offsets are migrated can be customized, and for added protection, MirrorMaker does not overwrite offsets if consumers on the target cluster are actively using the target consumer group, thus avoiding any accidental conflicts.

Topic configuration and ACL migration

In addition to mirroring data records, MirrorMaker may be configured to mirror topic configuration and access control lists (ACLs) of the topics to retain the same behavior for the mirrored topic. The default configuration enables this migration with reasonable periodic refresh intervals that may be sufficient in most cases. Most of the topic configuration settings from the source are applied to the target topic, but a few like `min.insync.replicas` are not applied by default. The list of excluded configs can be customized.

Only literal topic ACLs that match topics being mirrored are migrated, so if you are using prefixed or wildcard ACLs or alternative authorization mechanisms, you will need to configure those on the target cluster explicitly. ACLs for `Topic:Write` are not migrated to ensure that only MirrorMaker is allowed to write to the target topic. Appropriate access must be explicitly granted at the time of failover to ensure that applications work with the secondary cluster.

Connector tasks

The configuration option `tasks.max` limits the maximum number of tasks that the connector associated with MirrorMaker may use. The default is 1, but a minimum of 2 is recommended. When replicating a lot of topic partitions, higher values should be used if possible to increase parallelism.

Configuration prefixes

MirrorMaker supports customization of configuration options for all its components, including connectors, producers, consumers, and admin clients. Kafka Connect and connector configs can be specified without any prefix. But since MirrorMaker configuration can include configuration for multiple clusters, prefixes can be used to specify cluster-specific configs or configs for a particular replication flow. As we saw in the example earlier, clusters are identified using aliases that are used as a configuration prefix for options related to that cluster. Prefixes can be used to build a hierarchical configuration, with the more specific prefixed configuration having higher precedence than the less specific or nonprefixed configuration. MirrorMaker uses the following prefixes:

- `{cluster}.{connector_config}`
- `{cluster}.admin.{admin_config}`

- {source_cluster}.consumer.{consumer_config}
- {target_cluster}.producer.{producer_config}
- {source_cluster}->{target_cluster}.{replication_flow_config}

Multicluster Replication Topology

We have seen an example configuration for a simple active-standby replication flow for MirrorMaker. Now let's look at extending the configuration to support other common architectural patterns.

Active-active topology between New York and London can be configured by enabling replication flow in both directions. In this case, even though all topics from NYC are mirrored to LON and vice versa, MirrorMaker ensures that the same event isn't constantly mirrored back and forth between the pair of clusters since remote topics use the cluster alias as the prefix. It is good practice to use the same configuration file that contains the full replication topology for different MirrorMaker processes since it avoids conflicts when configs are shared using the internal configs topic in the target datacenter. MirrorMaker processes can be started in the target datacenter using the shared configuration file by specifying the target cluster when starting the MirrorMaker process using the `--clusters` option:

```
clusters = NYC, LON
NYC.bootstrap.servers = kafka.nyc.example.com:9092
LON.bootstrap.servers = kafka.lon.example.com:9092
NYC->LON.enabled = true                                ❶
NYC->LON.topics = .*                                    ❷
LON->NYC.enabled = true                                  ❸
LON->NYC.topics = .*                                    ❹
```

- ❶ Enable replication from New York to London.
- ❷ Specify topics that are replicated from New York to London.
- ❸ Enable replication from London to New York.
- ❹ Specify topics that are replicated from London to New York.

More replication flows with additional source or target clusters can also be added to the topology. For example, we can extend the configuration to support the fan out from NYC to SF and LON by adding a new replication flow for SF:

```
clusters = NYC, LON, SF
SF.bootstrap.servers = kafka.sf.example.com:9092
NYC->SF.enabled = true
NYC->SF.topics = .*
```

Securing MirrorMaker

For production clusters, it is important to ensure that all cross-datacenter traffic is secure. Options for securing Kafka clusters are described in [Chapter 11](#). MirrorMaker must be configured to use a secure broker listener in both source and target clusters, and client-side security options for each cluster must be configured for MirrorMaker to enable it to establish authenticated connections. SSL should be used to encrypt all cross-datacenter traffic. For example, the following configuration may be used to configure credentials for MirrorMaker:

```

NYC.security.protocol=SASL_SSL ❶
NYC.sasl.mechanism=PLAIN
NYC.sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule \
    required username="MirrorMaker" password="MirrorMaker-password"; ❷

```

- ❶ Security protocol should match that of the broker listener corresponding to the bootstrap servers specified for the cluster. `SASL` or `SASL_SSL` is recommended.
- ❷ Credentials for MirrorMaker are specified here using JAAS configuration since SASL is used. For SSL, keystores should be specified if mutual client authentication is enabled.

The principal associated with MirrorMaker must also be granted appropriate permissions on the source and target clusters if authorization is enabled on the clusters. ACLs must be granted for the MirrorMaker process for:

- `Topic:Read` on the source cluster to consume from source topics; `Topic:Create` and `Topic:Write` on the target cluster to create and produce to target topics.
- `Topic:DescribeConfigs` on the source cluster to obtain source topic configuration; `Topic:AlterConfigs` on the target cluster to update target topic configuration.
- `Topic:Alter` on the target cluster to add partitions if new source partitions are detected.
- `Group:Describe` on the source cluster to obtain source consumer group metadata, including offsets; `Group:Read` on the target cluster to commit offsets for those groups in the target cluster.
- `Cluster:Describe` on the source cluster to obtain source topic ACLs; `Cluster:Alter` on the target cluster to update the target topic ACLs.
- `Topic:Create` and `Topic:Write` permissions for internal MirrorMaker topics in the source and target clusters.

Deploying MirrorMaker in Production

In the previous example, we started MirrorMaker in dedicated mode on the command line. You can start any number of these processes to form a dedicated MirrorMaker cluster that is scalable and fault-tolerant. The processes mirroring to the same cluster will find each other and balance load between them automatically. Usually when running MirrorMaker in a production environment, you will want to run MirrorMaker as a service, running in the background with `nohup` and redirecting its console output to a log file. The tool also has `-daemon` as a command-line option that should do that for you. Most companies that use MirrorMaker have their own startup scripts that also include the configuration parameters they use. Production deployment systems like Ansible, Puppet, Chef, and Salt are often used to automate deployment and manage the many configuration options. MirrorMaker may also be run inside a Docker container. MirrorMaker is completely stateless and doesn't require any disk storage (all the data and state are stored in Kafka itself).

Since MirrorMaker is based on Kafka Connect, all deployment modes of Connect can be used with MirrorMaker. Standalone mode may be used for development and testing where MirrorMaker runs as a standalone Connect worker on a single machine. MirrorMaker may also be run as a connector in an existing distributed Connect cluster by explicitly configuring the connectors. For production use, we recommend running

MirrorMaker in distributed mode either as a dedicated MirrorMaker cluster or in a shared distributed Connect cluster.

If at all possible, run MirrorMaker at the target datacenter. So, if you are sending data from NYC to SF, MirrorMaker should run in SF and consume data across the US from NYC. The reason for this is that long-distance networks can be a bit less reliable than those inside a datacenter. If there is a network partition and you lose connectivity between the datacenters, having a consumer that is unable to connect to a cluster is much safer than a producer that can't connect. If the consumer can't connect, it simply won't be able to read events, but the events will still be stored in the source Kafka cluster and can remain there for a long time. There is no risk of losing events. On the other hand, if the events were already consumed and MirrorMaker can't produce them due to network partition, there is always a risk that these events will accidentally get lost by MirrorMaker. So, remote consuming is safer than remote producing.

When do you have to consume locally and produce remotely? The answer is when you need to encrypt the data while it is transferred between the datacenters but you don't need to encrypt the data inside the datacenter. Consumers take a significant performance hit when connecting to Kafka with SSL encryption—much more so than producers. This is because use of SSL requires copying data for encryption, which means consumers no longer enjoy the performance benefits of the usual zero-copy optimization. And this performance hit also affects the Kafka brokers themselves. If your cross datacenter traffic requires encryption, but local traffic does not, then you may be better off placing MirrorMaker at the source datacenter, having it consume unencrypted data locally, and then producing it to the remote datacenter through an SSL encrypted connection. This way, the producer connects to Kafka with SSL but not the consumer, which doesn't impact performance as much. If you use this consume locally and produce remotely approach, make sure MirrorMaker's Connect producer is configured to never lose events by configuring it with `acks=all` and a sufficient number of retries. Also, configure MirrorMaker to fail fast using `errors.tolerance=none` when it fails to send events, which is typically safer to do than to continue and risk data loss. Note that newer versions of Java have significantly increased SSL performance, so producing locally and consuming remotely may be a viable option even with encryption.

Another case where we may need to produce remotely and consume locally is a hybrid scenario when mirroring from an on-premises cluster to a cloud cluster. Secure on-premises clusters are likely to be behind a firewall that doesn't allow incoming connections from the cloud. Running MirrorMaker on premise allows all connections to be from on premises to the cloud.

When deploying MirrorMaker in production, it is important to remember to monitor it as follows:

Kafka Connect monitoring

Kafka Connect provides a wide range of metrics to monitor different aspects like connector metrics to monitor connector status, source connector metrics to monitor throughput, and worker metrics to monitor rebalance delays. Connect also provides a REST API to view and manage connectors.

MirrorMaker metrics monitoring

In addition to metrics from Connect, MirrorMaker adds metrics to monitor mirroring throughput and replication latency. The replication latency metric `replication-latency-ms` shows the time interval between the record timestamp and the time at which the record was successfully produced to the target cluster. This is useful to detect if the target is not keeping up with the source in a timely manner. Increased latency during peak hours may be OK if there is sufficient capacity to catch up later, but sustained increase in latency may indicate insufficient capacity. Other metrics like `record-age-ms`, which shows the age of records at the time of replication, `byte-rate`, which shows replication throughput, and `checkpoint-latency-ms`, which shows offset migration latency, can also be very useful. MirrorMaker also emits periodic heartbeats by default, which can be used to monitor its health.

Lag monitoring

You will definitely want to know if the target cluster is falling behind the source. The lag is the difference in offsets between the latest message in the source Kafka cluster and the latest message in the target cluster. See [Figure 10-7](#).

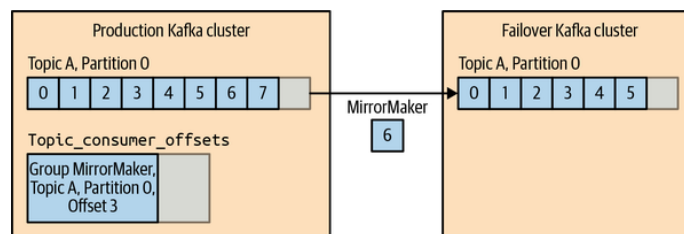


Figure 10-7. Monitoring the lag difference in offsets

In [Figure 10-7](#), the last offset in the source cluster is 7, and the last offset in the target is 5—meaning there is a lag of 2 messages.

There are two ways to track this lag, and neither is perfect:

- Check the latest offset committed by MirrorMaker to the source Kafka cluster. You can use the `kafka-consumer-groups` tool to check for each partition MirrorMaker is reading—the offset of the last event in the partition, the last offset MirrorMaker committed, and the lag between them. This indicator is not 100% accurate because MirrorMaker doesn't commit offsets all the time. It commits offsets every minute by default, so you will see the lag grow for a minute and then suddenly drop. In the diagram, the real lag is 2, but the `kafka-consumer-groups` tool will report a lag of 5 because MirrorMaker hasn't committed offsets for more recent messages yet. LinkedIn's Burrow monitors the same information but has a more sophisticated method to determine whether the lag represents a real problem, so you won't get false alerts.
- Check the latest offset read by MirrorMaker (even if it isn't committed). The consumers embedded in MirrorMaker publish key metrics in JMX. One of them is the consumer maximum lag (over all the partitions it is consuming). This lag is also not 100% accurate because it is updated based on what the consumer read but doesn't take into account whether the producer managed to send those messages to the destination Kafka cluster and whether they were acknowledged successfully. In this example, the MirrorMaker consumer will report a lag of 1 message rather than 2, because it already read message 6—even though the message wasn't produced to the destination yet.

Note that if MirrorMaker skips or drops messages, neither method will detect an issue because they just track the latest offset.

[Confluent Control Center](#) is a commercial tool that monitors message counts and checksums and closes this monitoring gap.

Producer and consumer metrics monitoring

The Kafka Connect framework used by MirrorMaker contains a producer and a consumer. Both have many available metrics, and we recommend collecting and tracking them. The [Kafka documentation](#) lists all the available metrics. Here are a few metrics that are useful in tuning MirrorMaker performance:

Consumer

`fetch-size-avg`, `fetch-size-max`, `fetch-rate`,
`fetch-throttle-time-avg`, and `fetch-throttle-time-max`

Producer

`batch-size-avg`, `batch-size-max`, `requests-in-flight`, and `record-retry-rate`

Both

`io-ratio` and `io-wait-ratio`

Canary

If you monitor everything else, a canary isn't strictly necessary, but we like to add it in for multiple layers of monitoring. It provides a process that, every minute, sends an event to a special topic in the source cluster and tries to read the event from the destination cluster. It also alerts you if the event takes more than an acceptable amount of time to arrive. This can mean that MirrorMaker is lagging or that it isn't available at all.

Tuning MirrorMaker

MirrorMaker is horizontally scalable. Sizing of the MirrorMaker cluster depends on the throughput you need and the lag you can tolerate. If you can't tolerate any lag, you have to size MirrorMaker with enough capacity to keep up with your top throughput. If you can tolerate some lag, you can size MirrorMaker to be 75–80% utilized 95–99% of the time. Then, expect some lag to develop when you are at peak throughput. Because MirrorMaker has spare capacity most of the time, it will catch up once the peak is over.

Then you want to measure the throughput you get from MirrorMaker with a different number of connector tasks—configured with the `tasks.max` parameter. This depends a lot on your hardware, datacenter, or cloud provider, so you will want to run your own tests. Kafka ships with the `kafka-performance-producer` tool. Use it to generate load on a source cluster and then connect MirrorMaker and start mirroring this load. Test MirrorMaker with 1, 2, 4, 8, 16, 24, and 32 tasks. Watch where performance tapers off and set `tasks.max` just below this point. If you are consuming or producing compressed events (recommended, since bandwidth is the main bottleneck for cross-datacenter mirroring), MirrorMaker will have to decompress and recompress the events. This uses a lot of CPU, so keep an eye on CPU utilization as you increase the number of tasks. Using this process, you will find the maximum throughput you can get with a single MirrorMaker worker. If it is not enough, you

will want to experiment with additional workers. If you are running MirrorMaker on an existing Connect cluster with other connectors, make sure you also take the load from those connectors into account when sizing the cluster.

In addition, you may want to separate sensitive topics—those that absolutely require low latency and where the mirror must be as close to the source as possible—to a separate MirrorMaker cluster. This will prevent a bloated topic or an out-of-control producer from slowing down your most sensitive data pipeline.

This is pretty much all the tuning you can do to MirrorMaker itself. However, you can still increase the throughput of each task and each MirrorMaker worker.

If you are running MirrorMaker across datacenters, tuning the TCP stack can help to increase the effective bandwidth. In Chapters 3 and 4, we saw that TCP buffer sizes can be configured for producers and consumers using `send.buffer.bytes` and `receive.buffer.bytes`. Similarly, broker-side buffer sizes can be configured using `socket.send.buffer.bytes` and `socket.receive.buffer.bytes` on brokers. These configuration options should be combined with optimization of the network configuration in Linux, as follows:

- Increase the TCP buffer size (`net.core.rmem_default` , `net.core.rmem_max` , `net.core.wmem_default` , `net.core.wmem_max` , and `net.core.optmem_max`)
- Enable automatic window scaling (`sysctl -w net.ipv4.tcp_window_scaling=1` or add `net.ipv4.tcp_window_scaling=1` to `/etc/sysctl.conf`)
- Reduce the TCP slow start time (set `/proc/sys/net/ipv4/tcp_slow_start_after_idle` to 0)

Note that tuning the Linux network is a large and complex topic. To understand more about these parameters and others, we recommend reading a network tuning guide such as *Performance Tuning for Linux Servers* by Sandra K. Johnson et al. (IBM Press).

In addition, you may want to tune the underlying producers and consumers of MirrorMaker. First, you will want decide whether the producer or the consumer is the bottleneck—is the producer waiting for the consumer to bring more data or the other way around? One way to decide is to look at the producer and consumer metrics you are monitoring. If one process is idle while the other is fully utilized, you know which one needs tuning. Another method is to do several thread dumps (using `jstack`) and see if the MirrorMaker threads are spending most of the time in poll or in send—more time spent polling usually means the consumer is the bottleneck, while more time spent sending shift points to the producer.

If you need to tune the producer, the following configuration settings can be useful:

linger.ms and batch.size

If your monitoring shows that the producer consistently sends partially empty batches (i.e., `batch-size-avg` and `batch-size-max` metrics are lower than configured `batch.size`), you can increase throughput by introducing a bit of latency. Increase `linger.ms` and the producer will wait a few milliseconds for the batches to fill up before sending them. If you are sending full batches and have

memory to spare, you can increase `batch.size` and send larger batches.

`max.in.flight.requests.per.connection`

Limiting the number of in-flight requests to 1 is currently the only way for MirrorMaker to guarantee that message ordering is preserved if some messages require multiple retries before they are successfully acknowledged. But this means every request that was sent by the producer has to be acknowledged by the target cluster before the next message is sent. This can limit throughput, especially if there is significant latency before the brokers acknowledge the messages. If message order is not critical for your use case, using the default value of 5 for

`max.in.flight.requests.per.connection` can significantly increase your throughput.

The following consumer configurations can increase throughput for the consumer:

`fetch.max.bytes`

If the metrics you are collecting show that `fetch-size-avg` and `fetch-size-max` are close to the `fetch.max.bytes` configuration, the consumer is reading as much data from the broker as it is allowed. If you have available memory, try increasing `fetch.max.bytes` to allow the consumer to read more data in each request.

`fetch.min.bytes` and `fetch.max.wait.ms`

If you see in the consumer metrics that `fetch-rate` is high, the consumer is sending too many requests to the brokers and not receiving enough data in each request. Try increasing both `fetch.min.bytes` and `fetch.max.wait.ms` so the consumer will receive more data in each request and the broker will wait until enough data is available before responding to the consumer request.

Other Cross-Cluster Mirroring Solutions

We looked in depth at MirrorMaker because this mirroring software arrives as part of Apache Kafka. However, MirrorMaker also has some limitations when used in practice. It is worthwhile to look at some of the alternatives to MirrorMaker and the ways they address MirrorMaker limitations and complexities. We describe a couple of open source solutions from Uber and LinkedIn and commercial solutions from Confluent.

Uber uReplicator

Uber ran legacy MirrorMaker at very large scale, and as the number of topics and partitions grew and the cluster throughput increased, it started running into several problems. As we saw earlier, the legacy MirrorMaker used consumers that were members of a single consumer group to consume from source topics. Adding MirrorMaker threads, adding MirrorMaker instances, bouncing MirrorMaker instances, or even adding new topics that match the regular expression used in the inclusion filter all caused consumers to rebalance. As we saw in [Chapter 4](#), rebalancing stops all the consumers until new partitions can be assigned to

each consumer. With a very large number of topics and partitions, this can take a while. This is especially true when using old consumers like Uber did. In some cases, this caused 5–10 minutes of inactivity, causing mirroring to fall behind and accumulate a large backlog of events to mirror, which can take a long time to recover from. This caused very high latency for consumers reading events from the destination cluster. To avoid rebalances when someone added a topic matching the topic inclusion filter, Uber decided to maintain a list of exact topic names to mirror instead of using a regular expression filter. But this was hard to maintain as all MirrorMaker instances had to be reconfigured and bounced to add a new topic. If not done correctly, this could result in endless rebalances as the consumers won't be able to agree on the topics they subscribe to.

Given these issues, Uber decided to write its own MirrorMaker clone, called `uReplicator`. Uber decided to use Apache Helix as a central (but highly available) controller to manage the topic list and the partitions assigned to each `uReplicator` instance. Administrators use a REST API to add new topics to the list in Helix, and `uReplicator` is responsible for assigning partitions to the different consumers. To achieve this, Uber replaced the Kafka consumers used in MirrorMaker with a Kafka consumer Uber engineers wrote called Helix consumer. This consumer takes its partition assignment from the Apache Helix controller rather than as a result of an agreement between the consumers (see [Chapter 4](#) for details on how this is done in Kafka). As a result, the Helix consumer can avoid rebalances and instead listen to changes in the assigned partitions that arrive from Helix.

Uber engineering wrote a [blog post](#) describing the architecture in more detail and showing the improvements they experienced. `uReplicator`'s dependency on Apache Helix introduces a new component to learn and manage, adding complexity to any deployment. As we saw earlier, MirrorMaker 2.0 solves many of these scalability and fault-tolerance issues of legacy MirrorMaker without any external dependencies.

LinkedIn Brooklin

Like Uber, LinkedIn was also using legacy MirrorMaker for transferring data between Kafka clusters. As the scale of the data grew, it also ran into similar scalability issues and operational challenges. So LinkedIn built a mirroring solution on top of its data streaming system called Brooklin. Brooklin is a distributed service that can stream data between different heterogeneous data source and target systems, including Kafka. As a generic data ingestion framework that can be used to build data pipelines, Brooklin supports multiple use cases:

- Data bridge to feed data into stream processing systems from different data sources
- Stream change data capture (CDC) events from different data stores
- Cross-cluster mirroring solution for Kafka

Brooklin is a scalable distributed system designed for high reliability and has been tested with Kafka at scale. It is used to mirror trillions of messages a day and has been optimized for stability, performance, and operability. Brooklin comes with a REST API for management operations. It is a shared service that can process a large number of data pipelines, enabling the same service to mirror data across multiple Kafka clusters.

Confluent Cross-Datacenter Mirroring Solutions

At the same time that Uber developed its uReplicator, Confluent independently developed Confluent Replicator. Despite the similarities in names, the projects have almost nothing in common—they are different solutions to two different sets of MirrorMaker problems. Like MirrorMaker 2.0, which came later, Confluent’s Replicator is based on the Kafka Connect framework and was developed to address issues its enterprise customers encountered when using legacy MirrorMaker to manage their multiclust deployments.

For customers who use stretch clusters for their operational simplicity and low RTO and RPO, Confluent added Multi-Region Cluster (MRC) as a built-in feature of Confluent Server, which is a commercial component of the Confluent Platform. MRC extends Kafka’s support for stretch clusters using asynchronous replicas to limit impact on latency and throughput. Like stretch clusters, this is suitable for replication between availability zones or regions with latencies less than 50 ms and benefits from transparent client failover. For distant clusters with less reliable networks, a new built-in feature called Cluster Linking was added to Confluent Server more recently. Cluster Linking extends Kafka’s offset-preserving intra-cluster replication protocol to mirror data between clusters.

Let’s look at the features supported by each of these solutions:

Confluent Replicator

Confluent Replicator is a mirroring tool similar to MirrorMaker that relies on the Kafka Connect framework for cluster management and can run on existing Connect clusters. Both support data replication for different topologies as well as migration of consumer offsets and topic configuration. There are some differences in features between the two. For example, MirrorMaker supports ACL migration and offset translation for any client, but Replicator doesn’t migrate ACLs and supports offset translation (using timestamp interceptor) only for Java clients. Replicator doesn’t have the concept of local and remote topics like MirrorMaker, but it supports aggregate topics. Like MirrorMaker, Replicator also avoids replication cycles but does so using provenance headers. Replicator provides a range of metrics, like replication lag, and can be monitored using its REST API or Control Center UI. It also supports schema migration between clusters and can perform schema translation.

Multi-Region Clusters (MRC)

We saw earlier that stretch clusters provide simple transparent failover and fallback for clients without the need for offset translation or client restarts. But stretch clusters require datacenters to be close to each other and provide a stable low-latency network to enable synchronous replication between datacenters. MRC is also suitable only for datacenters within a 50 ms latency, but it uses a combination of synchronous and asynchronous replication to limit impact on producer performance and provide higher network tolerance.

As we saw earlier, Apache Kafka supports fetching from followers to enable clients to fetch from their closest brokers based on rack ID, thereby reducing cross-datacenter traffic. Confluent Server also adds the concept of *observers*, which are asynchronous replicas that do not join the ISR and hence have no impact on producers us-

ing `acks=all` but are able to deliver records to consumers. Operators can configure synchronous replication within a region and asynchronous replication between regions to benefit from both low latency and high durability at the same time. Replica placement constraints in Confluent Server allow you to specify a minimum number of replicas per region using rack IDs to ensure that replicas are spread across regions to guarantee durability. Confluent Platform 6.1 also adds automatic observer promotion with configurable criteria, enabling fast failover without data loss automatically. When `min.insync.replicas` falls below a configured minimum number of synchronous replicas, observers that have caught up are automatically promoted to allow them to join ISRs, bringing the number of ISRs back up to the required minimum. The promoted observers use synchronous replication and may impact throughput, but the cluster remains operational throughout without data loss even if a region fails. When the failed region recovers, observers are automatically demoted, getting the cluster back to normal performance levels.

Cluster Linking

Cluster Linking, introduced as a preview feature in Confluent Platform 6.0, builds inter-cluster replication directly into the Confluent Server. By using the same protocol as inter-broker replication within a cluster, Cluster Linking performs offset-preserving replication across clusters, enabling seamless migration of clients without any need for offset translation. Topic configuration, partitions, consumer offsets, and ACLs are all kept synchronized between the two clusters to enable failover with low RTO if a disaster occurs. A cluster link defines the configuration of a directional flow from a source cluster to a destination cluster. Leader brokers of mirror partitions in the destination cluster fetch partition data from the corresponding source leaders, while followers in the destination replicate from their local leader using the standard replication mechanism in Kafka. Mirror topics are marked as read-only in the destination to prevent any local produce to these topics, ensuring that mirror topics are logically identical to their source topic.

Cluster Linking provides operational simplicity without the need for separate clusters like Connect clusters and is more performant than external tools since it avoids decompression and recompression during mirroring. Unlike MRC, there is no option for synchronous replication, and client failover is a manual process that requires client restart. But Cluster Linking may be used with distant datacenters with unreliable high-latency networks and reduces cross-datacenter traffic by replicating only once between datacenters. It is suitable for cluster migration and topic sharing use cases.

Summary

We started the chapter by describing the reasons you may need to manage more than a single Kafka cluster and then proceeded to describe several common multicluster architectures, ranging from the simple to the very complex. We went into the details of implementing failover architecture for Kafka and compared the different options currently available. Then we proceeded to discuss the available tools. Starting with Apache Kafka's MirrorMaker, we went into many details of using it in production.

We finished by reviewing alternative options that solve some of the issues you might encounter with MirrorMaker.

Whichever architecture and tools you end up using, remember that multicluster configuration and mirroring pipelines should be monitored and tested just like everything else you take into production. Because multicluster management in Kafka can be easier than it is with relational databases, some organizations treat it as an afterthought and neglect to apply proper design, planning, testing, deployment automation, monitoring, and maintenance. By taking multicluster management seriously, preferably as part of a holistic disaster or geodiversity plan for the entire organization that involves multiple applications and data stores, you will greatly increase the chances of successfully managing multiple Kafka clusters.